

IC 工程师必备：Git 版本控制与“后悔药”秘籍

Huang & Gemini 联合出品

2026 年 7 月 14 日

1 引言

在数字芯片大项目协同开发中，Git 是管理数万行 Verilog 代码、防止代码冲突的绝对神器。本章专门总结 Git 的高频核心命令，并将其映射到 Windows 图形客户端（如 TortoiseGit 或 VS Code Git 插件）的对应操作。

2 Git 核心工作区概念

Git 的精髓在于三大空间的转换：

1. **工作区 (Working Directory)**：你正在用 Vim 修改代码的本地文件夹。
 2. **暂存区 (Stage/Index)**：准备提交的临时“待办箱”。
 3. **本地仓库 (Repository)**：已经盖章保存的“历史版本库”（每一个 Commit 都是一个无法篡改的时空存档）。
-

3 Git 核心命令与 GUI 操作对照表

4 Git 经典工作流（极简三连击）

在实际开发中，你修改完 Verilog 代码并测试通过后，通常会一次性敲下这三行命令完成代码备份：

```
# 1. 确认当前改动了哪些文件
git status

# 2. 把修改好的文件塞进暂存区
git add alu_top.v
```

表 1: Git 命令行 vs Windows/GUI 对应关系

Git 命令行	Windows / TortoiseGit 等价操作	实际作用
<code>git init</code>	右键空白处 → <i>Git Create Repository here...</i>	在当前文件夹初始化一个本地 Git 仓库（生成隐藏的 <code>.git</code> 文件夹）
<code>git status</code>	右键 → <i>Git Commit...</i> 弹出的窗口列表	查看哪些文件被修改了、哪些是还没追踪的新文件
<code>git add alu_top.v</code>	在 Commit 窗口中, 勾选 <code>alu_top.v</code> 前面的复选框	将修改后的代码放入“暂存区”（装箱准备寄出）
<code>git commit -m "add rst"</code>	在 Commit 窗口输入日志, 点击下方的 [Commit] 按钮	将暂存区的内容正式提交, 生成一个永久的历史版本快照
<code>git log</code>	右键 → <i>TortoiseGit</i> → <i>Show Log</i>	查看所有提交的历史记录（谁、在什么时间、改了什么）
<code>git diff alu_top.v</code>	双击修改过的文件 → 弹出对比窗口 (Compare)	对比当前代码和上一个版本具体有哪些行被改动了
<code>git restore alu_top.v</code>	右键文件 → <i>TortoiseGit</i> → <i>Revert...</i>	吃后悔药 : 彻底放弃当前还未提交的修改, 一键还原代码

3. 提交版本，并写好备注信息

```
git commit -m "Design: complete the reset logic for alu_top"
```